Customer Support Reply Prompts - Project Report

Designing and testing a prompt that sorts customer emails and drafts replies that follow store policy

EXAMPLE REPORT. This shows the sections and the level of detail the Capstone agent expects. Your own .docx report needs exactly three sections: **Problem Statement**, **Approach** and **Conclusion**. Do not paste your prompts, model outputs or screenshots into it: the prompts, test cases, saved responses, evaluation script and screenshots all go in the .zip, the screenshots in its **output_screenshots** folder. Write about your own project; do not copy this one.

Problem Statement

Nila Home Store, a hypothetical online home-goods shop, gets the same few kinds of customer email every day: damaged items, late deliveries, refund requests and so on. Support staff spend most of their time writing similar replies. The goal is a prompt that makes a language model put each email in the right category and draft a reply that staff can send after a quick check.

The prompt has to meet these requirements:

- Put each email in exactly one of eight categories.
- Return JSON, so the support tool can read the category and the reply.
- Never promise a refund the store's policy does not allow.
- Keep replies under 120 words, starting with "Hi," and ending with the team's sign-off.
- Show, with measurements, that each change to the prompt made it better.

Approach

A fixed test set and automatic checks

Before writing any prompt, I wrote 8 test emails, one per category, each with its correct category and whether a refund is allowed. Three are traps: a refund request outside the 30-day window, an exchange that sounds like a refund, and a complaint about a courier that mentions delivery. Every response is scored by 5 rule-based checks in Python (valid JSON, correct category, refund-safe, length, greeting and sign-off), so every version is judged the same way. The responses are saved, so anyone can re-check the scores without calling the model again.

Three prompt versions

- **v1 baseline**: a one-line instruction, to see what the model does with no guidance.
- **v2 role and rules**: a support-agent role, the eight categories, the refund policy and a request for JSON.

- **v3 few-shot and strict JSON**: the policy as an explicit list, a rule for the courier-complaint case, the exact output format, and two worked examples, one showing how to turn down a refund politely.

Each version was run once per test email at temperature 0.2, and the raw responses were saved.

Conclusion

Version	JSON	Category	Refund-safe	Length	Format	Overall
v1 baseline	0/8	0/8	6/8	7/8	0/8	32.5%
v2 role and rules	7/8	5/8	7/8	8/8	0/8	67.5%
v3 few-shot and strict JSON	8/8	8/8	8/8	8/8	7/8	97.5%

The final prompt scores 97.5%, up from 32.5%. Version 1 wrote friendly prose but promised refunds the policy does not allow. Version 2 fixed the format, but still mentioned a refund to the 45-day customer, called the size exchange a refund request and once put text before the JSON. The worked examples in version 3 fixed all three. The one remaining failure is a sign-off with an extra word ("Support Team"). All five requirements are met.

What I learned

Worked examples changed the model's behaviour more than extra rules did, especially for saying no politely. Writing the test cases first meant every change could be measured instead of judged by reading a few replies.

Limitations and possible improvements

- Eight emails is a small test set; real emails in Tamil, Hindi and mixed languages should be added.
- Each prompt was run once; running each case several times would show how consistent it is.
- The checks cannot judge tone or accuracy, so a person should still review a sample of replies.